SIMATSENGINEERING

LABEXPERIMENTS

Student Name: A.Bhavya

Reg No: 192424417

Course Code: CSA0613

Subject Name: Design and Analysis of Algorithms

TOPIC-1 INTRODUCTION

1. Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward. Example 1: Input: words = ["abc","car","ada","racecar","cool"] Output: "ada" Explanation: The first string that is palindromic is "ada". Note that "racecar" is also palindromic, but it is not the first. Example 2: Input: words = ["notapalindrome","racecar"] Output: "racecar" Explanation: The first and only string that is palindromic is "racecar".

Aim

To find and return the first palindromic string from a given array of strings. If none exists, return an empty string.

Algorithm

1. Start
2. Take input array of strings
3. For each string in the array:
    - Reverse the string
    - Compare original string with reversed string
    - If both are same → return that string immediately
4. If no palindrome found, return empty string ""
5. Stop

Code

```
def first_palindrome(words):
    for word in words:
        if word == word[::-1]:
            return word
    return ""

words = ["abc","car","ada","racecar","cool"]
result = first_palindrome(words)
print(result)
```

Input words =

["abc","car","ada","racecar","cool"]

Output

Result

The program successfully identifies the first palindrome ("ada") in the array and returns it.

2. You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2]. Example 1: Input: nums1 = [2,3,2], nums2 = [1,2] Output:

[2,1] Explanation: Example 2: Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6] Output: [3,4] Explanation: The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3. The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.

Aim

To count how many elements of one array exist in another array and return both counts.

Algorithm

1. Start
2. Read arrays nums1 and nums2
3. Initialize answer1 and answer2 as 0
4. Traverse nums1
5. If element exists in nums2, increment answer1
6. Traverse nums2
7. If element exists in nums1, increment answer2
8. Print [answer1, answer2]
9. Stop

Code nums1 =

[2,3,2] nums2 =

[1,2] answer1 =

0 answer2 = 0

```
for i in nums1:
    if i in nums2:
```

```
        answer1 += 1
for i in nums2:
    if i in nums1:
        answer2 += 1
print([answer1, answer2])
```

Input nums1 =

[2,3,2] nums2 =

[1,2]

Output

Result

The program successfully counts the matching elements between the two arrays.

3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array. Example 1: Input: nums = [1,2,1] Output: 15 Explanation: Six possible

subarrays are: [1]: 1 distinct value [2]: 1 distinct value [1]: 1 distinct value [1,2]: 2 distinct values [2,1]: 2 distinct values [1,2,1]: 2 distinct values The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 + 22 + 22 + 22 = 15. Example 2: Input: nums = [1,1] Output: 3 Explanation: Three possible subarrays are: [1]: 1 distinct value [1]: 1 distinct value [1,1]: 1 distinct value The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 = 3.

Aim

To calculate the sum of squares of distinct element counts of all subarrays.

Algorithm

1. Start
2. Read array nums
3. Generate all possible subarrays
4. Find distinct elements in each subarray
5. Square the distinct count
6. Add to total sum
7. Print total sum
8. Stop **Code** nums = [1,2,1] result = 0

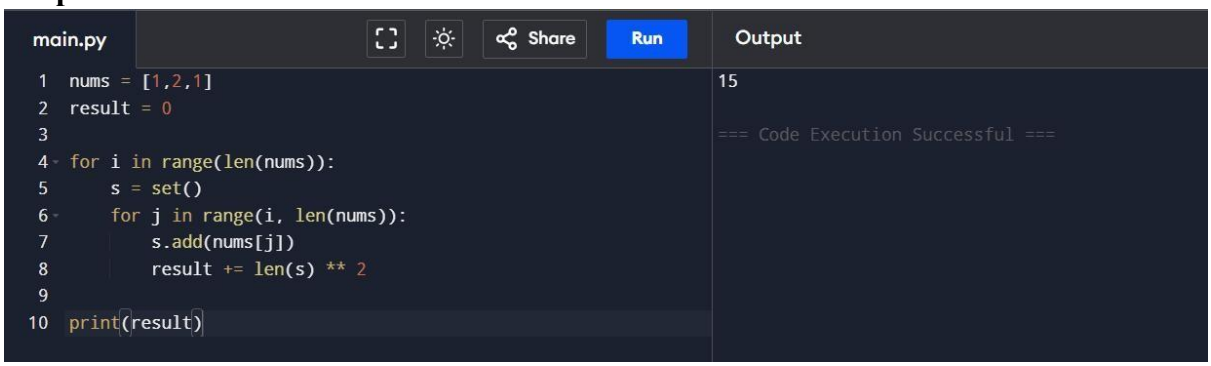
```
for i in range(len(nums)): s =
   set() for j in range(i,
   len(nums)):
      s.add(nums[j])
      result += len(s) **
      2
```

print(result)

Input nums =

[1,2,1]

Output



Result

The program successfully calculates the sum of squares of distinct counts of all subarrays.

4. Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by k. Example 1: Input: nums = [3,1,2,2,2,1,3], k = 2 Output: 4 Explanation: There are 4 pairs that meet all the requirements: - nums[0] == nums[6], and 0 * 6 == 0, which is divisible by 2. - nums[2] == nums[3], and 2 * 3 == 6, which is divisible by 2. nums[2] == nums[4], and 2 * 4 == 8, which is divisible by 2. - nums[3] == nums[4], and 3 * 4 == 12, which is divisible by 2. Example 2: Input: nums = [1,2,3,4], k = 1 Output: 0 Explanation: Since no value in nums is repeated, there are no pairs (i,j) that meet all the requirements.

Aim

To find the number of pairs where nums[i] == nums[j] and (i*j) is divisible by k.

Algorithm

1. Start
2. Read nums and k
3. Initialize count as 0

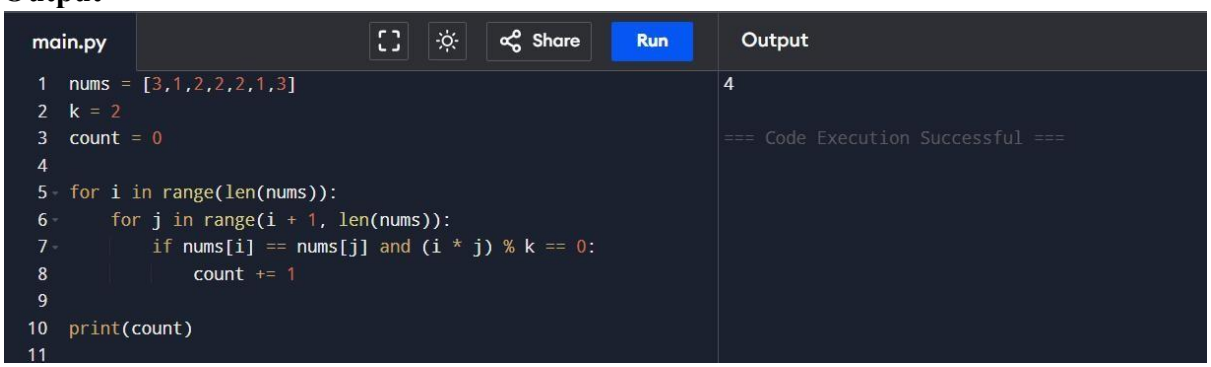
4. Traverse all pairs
5. Check equality and divisibility condition
6. Increment count if true
7. Print count
8. Stop **Code**

```
nums = [3,1,2,2,2,1,3]

k = 2

count = 0


for i in range(len(nums)): for j in

    range(i + 1, len(nums)):


        if nums[i] == nums[j] and (i * j) % k == 0:

            count += 1

print(count)
```

Input nums = [3,1,2,2,2,1,3]

, k = 2

Output



Result

The program successfully finds valid pairs satisfying the conditions.

5. Write a program FOR THE BELOW TEST CASES with least time complexity Test Cases: - Input: {1, 2, 3, 4, 5} Expected Output: 5 Input: {7, 7, 7, 7, 7} Expected Output: 7 Input: {-10, 2, 3, -4, 5} Expected Output: 5

Aim

To find the maximum element in an array.

Algorithm

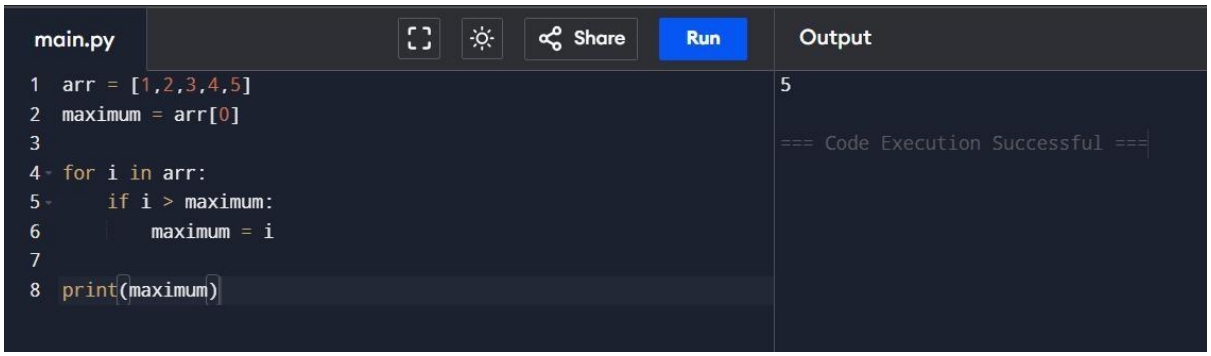
1. Start
2. Read array
3. Assume first element as maximum
4. Compare with remaining elements
5. Update maximum if larger element found
6. Print maximum element
7. Stop **Code**

```
arr = [1,2,3,4,5]
maximum = arr[0]
for i in arr:
    if i > maximum:
        maximum = i
print(maximum)
```

Input arr =

[1,2,3,4,5]

Output



Result

The program successfully finds the maximum element in the array.

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same. Test Cases 1. Empty List 1. Input: [] 2. Expected Output: None or an appropriate message indicating that the list is empty. 2. Single Element List 1. Input: [5] 2. Expected Output: 5 3. All Elements are the Same 1. Input: [3, 3, 3, 3, 3] 2. Expected Output: 3

Aim

To sort a list and find the maximum element.

Algorithm

1. Start
2. Read list
3. Sort the list
4. Find last element as maximum
5. Print maximum element
6. Stop **Code**

arr = [3,1,5,2,4]

if len(arr) == 0:

```
    print("List is empty")
else:
    arr.sort()
print(arr[-1])
```

Input arr =

[3,1,5,2,4]

Output

Result

The program successfully sorts the list and finds the maximum element.

7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm? Test Cases Some Duplicate Elements Input: [3, 7, 3, 5, 2, 5, 9, 2] Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used) Negative and Positive Numbers Input: [-1, 2, -1, 3, 2, -2] Expected Output: [-1, 2, 3, -2] (Order may vary) List with Large Numbers Input: [1000000, 999999, 1000000] Expected Output: [1000000, 999999]

Aim

To create a list containing only unique elements.

Algorithm

1. Start

2. Read list
3. Create empty list for unique elements
4. Traverse original list
5. Add element if not already present
6. Print unique list
7. Stop **Code**

```
arr = [3,7,3,5,2,5,9,2]
unique = [] for i in
arr:
    if i not in unique:
        unique.append(i)
print(unique)
```

Input arr =

[3,7,3,5,2,5,9,2]

Output

Result

The program successfully removes duplicate elements from the list.

8.Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

Aim

To sort an array using bubble sort technique.

Algorithm

1. Start
2. Read array
3. Compare adjacent elements
4. Swap if left element is greater
5. Repeat until sorted
6. Print sorted array
7. Stop

Code

```
arr = [5,1,4,2,8] for i in

range(len(arr)): for j in

range(0, len(arr)-i-1):

    if arr[j] > arr[j+1]:

        arr[j], arr[j+1] = arr[j+1], arr[j]

print(arr)
```

Input arr =

[5,1,4,2,8]

Output

Result

The program successfully sorts the array using bubble sort.

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation. Test Case: Example X={ 3,4,6,-9,10,8,9,30} KEY=10 Output: Element 10 is found at position 5 Example X={ 3,4,6,-9,10,8,9,30} KEY=100 Output : Element 100 is not found.

Aim

To search an element in a sorted array using binary search.

Algorithm

1. Start
2. Read sorted array and key
3. Find middle element
4. Compare key with middle element
5. If equal, print position
6. If smaller, search left half
7. If larger, search right half
8. Repeat until found or search ends

9. Stop **Code**

```
arr = [-9,3,4,6,8,9,10,30]

key = 10

low = 0

high = len(arr) - 1

found = False

while low <= high:

    mid = (low + high) // 2

    if arr[mid] == key: print("Element found at

    position", mid + 1) found = True break elif

    arr[mid] < key:

        low = mid + 1

    else:

        high = mid - 1

if not found:

    print("Element not found")
```

Input arr = [-

9,3,4,6,8,9,10,30]

key = 10

Output

main.py

Share Run

```
arr = [-9,3,4,6,8,9,10,30]
key = 10
low = 0
high = len(arr) - 1
found = False
while low <= high:
    mid = (low + high) // 2
    if arr[mid] == key:
        print("Element found at position", mid + 1)
        found = True
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
if not found:
    print("Element not found")
```

Output

```
Element found at position 7

=== Code Execution Successful ===
```

Result

The program successfully searches the element using binary search.

10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

Aim

To sort an array in ascending order using merge sort.

Algorithm

1. Divide array into halves
2. Recursively sort both halves
3. Merge sorted halves and Print sorted array. **Code**

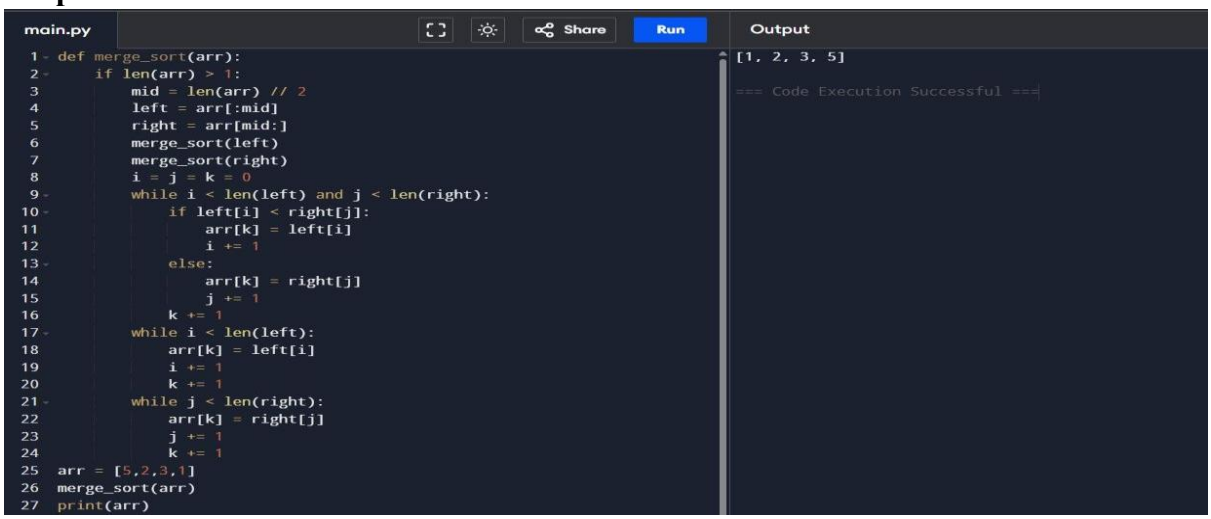
```
def merge_sort(arr):
    if len(arr) > 1: mid
        = len(arr) // 2 left
        = arr[:mid] right
        = arr[mid:]
```

```
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i  += 1
            else:
                arr[k] = right[j]
                j  += 1
                k += 1
        while i < len(left):
            arr[k] = left[i]
            i          +
            = 1 k
            += 1
        while j < len(right):
            arr[k] = right[j]
            j          +
            = 1 k
            += 1
arr = [5,2,3,1]
merge_sort(arr)
print(arr)
```

Input arr =

[5,2,3,1]

Output



Result

The program successfully sorts the array using merge sort.

EXERCISE 2

11. Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps. Example: · Input: m=2,n=2,N=2,i=0,j=0 · Output: 6 · Input: m=1,n=3,N=3,i=0,j=1 · Output: 12

Aim

To find the number of ways to move the ball out of the grid boundary in exactly N steps.

Algorithm

1. Start
2. Use recursion with memoization
3. If ball moves outside boundary, return 1
4. If no moves left, return 0
5. Move in all four directions

6. Add all possible ways

7. Print result

8. Stop **Code** from functools import lru_cache

```
m = 2

n  = 2

N = 2

startRow = 0

startColumn = 0

@lru_cache(None)

def dfs(i, j, moves): if

i < 0 or i >= m or j <

0 or j >= n:

    return 1 if

  moves == 0:

    return 0 return ( dfs(i +

  1, j, moves - 1) + dfs(i - 1,

  j, moves - 1) + dfs(i, j + 1,

  moves - 1) + dfs(i, j - 1,

  moves - 1)

  )

print(dfs(startRow, startColumn, N))
```

Input

```
m = 2

n  = 2
```

N = 2

startRow = 0

startColumn = 0

Output

main.py

```
from functools import lru_cache
m = 2
n = 2
N = 2
startRow = 0
startColumn = 0
@lru_cache(None)
def dfs(i, j, moves):
    if i < 0 or i >= m or j < 0 or j >= n:
        return 1
    if moves == 0:
        return 0
    return (
        dfs(i + 1, j, moves - 1) +
        dfs(i - 1, j, moves - 1) +
        dfs(i, j + 1, moves - 1) +
        dfs(i, j - 1, moves - 1)
    )
print(dfs(startRow, startColumn, N))
```

Output

```
6

=== Code Execution Successful ===
```

Result

The program successfully calculates the number of ways to move the ball out of the grid.

12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night. Examples: Input : nums = [2, 3, 2] Output : The maximum money you can rob without alerting the police is 3(robbing house 1). (ii)Input : nums = [1, 2, 3, 1] Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

Aim

To find the maximum amount of money that can be robbed without robbing adjacent houses arranged in a circle.

Algorithm

1. Start

2. Exclude first house and solve
3. Exclude last house and solve
4. Find maximum of both cases
5. Print result
6. Stop **Code**

```
def rob_linear(arr):
    prev = curr = 0
    for num in arr:
        prev, curr = curr, max(curr, prev + num)
    return curr

nums = [2,3,2] if
len(nums) == 1:
    print(nums[0])
else:
    print(max(rob_linear(nums[:-1]), rob_linear(nums[1:])))
```

Input nums =

[2,3,2]

Output

Result

The program successfully finds the maximum money that can be robbed.

13. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top? Examples: Input: n=4 Output: 5 Input: n=3 Output: 3

Aim

To find the number of distinct ways to climb to the top.

Algorithm

1. Start
2. Read number of stairs n
3. Use Fibonacci relation
4. Calculate ways iteratively
5. Print result
6. Stop **Code** n = 4

if n <= 2:

print(n)

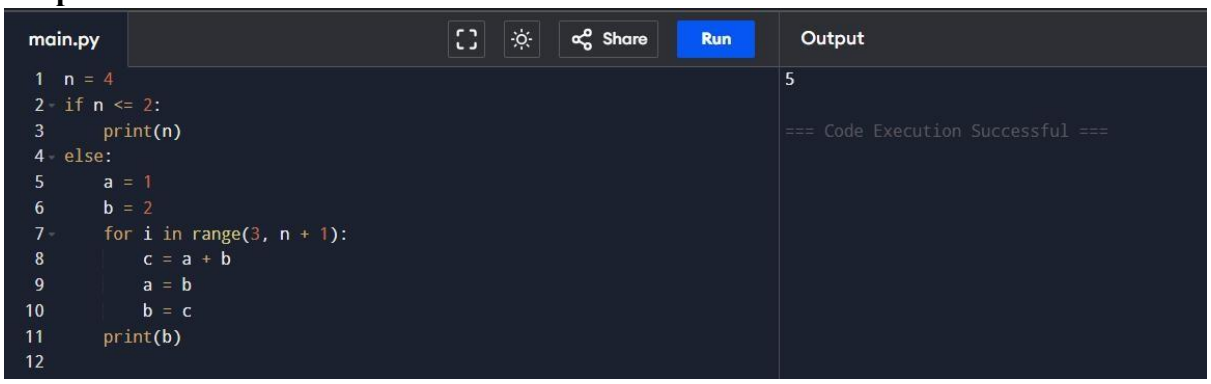
else:

```
    a = 1
    b = 2
    for i in range(3, n + 1):
        c = a +
        b a = b
        b = c
    print(b)
```

Input

n = 4

Output



Result

The program successfully calculates the number of ways to climb stairs.

14. A robot is located at the top-left corner of a m×n grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there? Examples: Input: m=7,n=3 Output: 28 Input: m=3,n=2 Output: 3.

Aim

To find the number of unique paths from top-left to bottom-right of a grid.

Algorithm

1. Start
2. Create DP table
3. Fill first row and column with 1
4. Add top and left values for remaining cells
5. Print bottom-right value
6. Stop

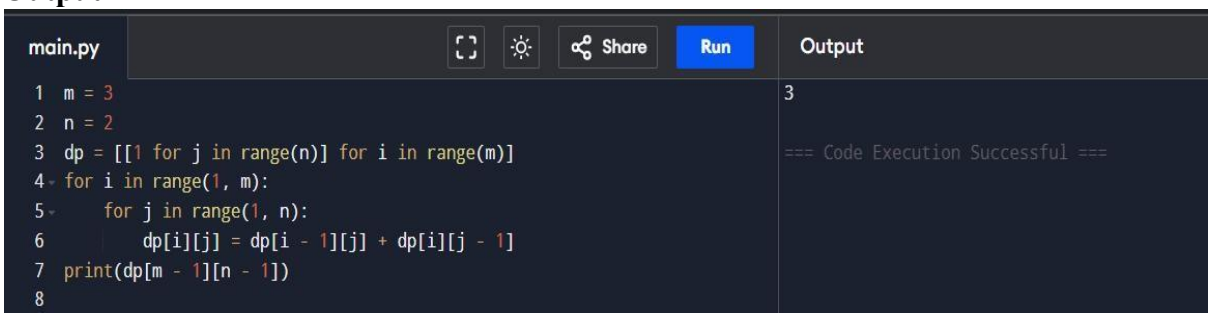
Code

```
m = 3

n = 2

dp = [[1 for j in range(n)] for i in range(m)]

for i in range(1, m):

    for j in range(1, n):

        dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

print(dp[m - 1][n - 1])
```

Input

m = 3 , n = 2

Output



Result

The program successfully finds the number of unique paths.

14. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index. Example 1: Input: s = "abbxxxxzzy" Output: [[3,6]] Explanation: "xxxx" is the only large group with start index 3 and end index 6. Example 2: Input: s = "abc" Output: [] Explanation: We have groups "a", "b", and "c", none of which are large groups.

Aim

To find intervals of large groups in a string.

Algorithm

1. Start
2. Traverse string
3. Count consecutive repeated characters
4. If count >= 3, store interval
5. Print intervals
6. Stop

Code s = "abbxxxxzzy" result = []

start = 0 for i in range(len(s)): if i

== len(s) - 1 or s[i] != s[i + 1]: if i -

start + 1 >= 3: result.append([start,

i])

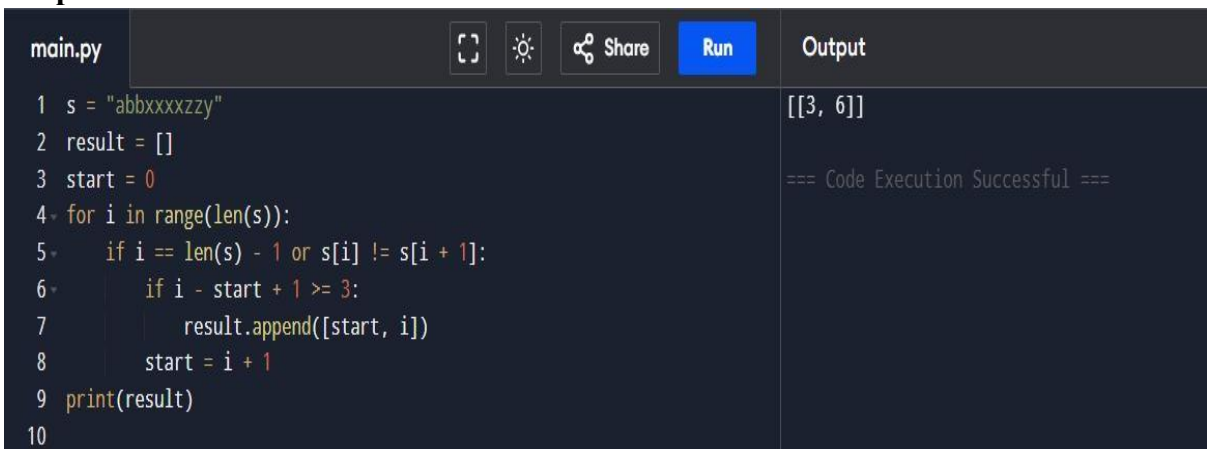
start = i + 1

print(result)

Input s =

"abbxxxxzzy"

Output



Result

The program successfully finds all large group intervals.

15. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules Any live cell with fewer than two live neighbors dies as if caused by under-population. Any live cell with two or three live neighbors lives on to the next generation. Any live cell with more than three live neighbors dies, as if by

over-population. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction. The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the m x n grid board, return the next state. Example 1: Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]] Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]] Example 2: Input: board = [[1,1],[1,0]] Output: [[1,1],[1,1]]

Aim

To find the next state of the board using Game of Life rules.

Algorithm

1. Start
2. Traverse each cell
3. Count live neighbors
4. Apply Game of Life rules
5. Store updated board
6. Print next state
7. Stop

Code

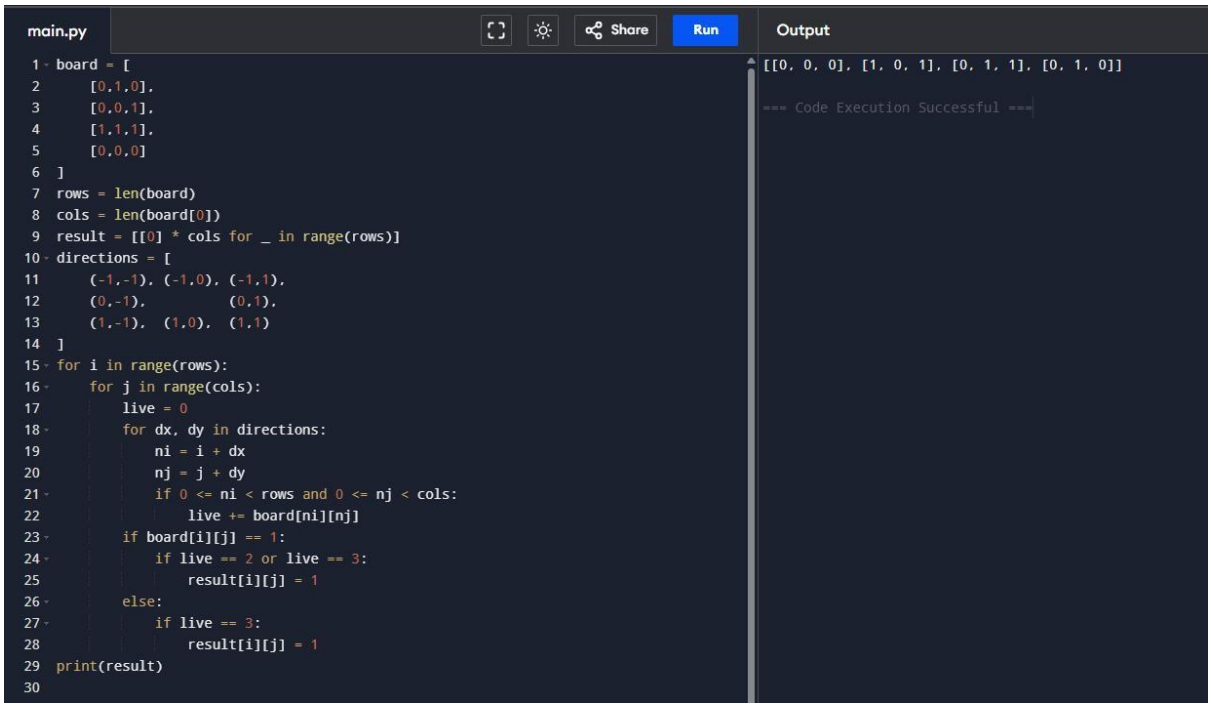
```
board = [
    [0,1,0],
    [0,0,1],
    [1,1,1],
    [0,0,0]
]
rows = len(board)
```

```
cols = len(board[0]) result = [[0] * cols

for _ in range(rows)] directions = [

    (-1,-1), (-1,0), (-1,1),

    (0,-1),          (0,1),

    (1,-1), (1,0), (1,1)

]

for i in range(rows):

    for j in range(cols): live =

        0 for dx, dy in

        directions:

            ni = i + dx nj = j + dy if 0 <= ni <

            rows and 0 <= nj < cols:

                live += board[ni][nj]

        if board[i][j] == 1:

            if live == 2 or live == 3:

                result[i][j] = 1

        else:

            if live == 3:

                result[i][j] = 1

print(result)
```

Input

```
board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
```

Output



Result

The program successfully generates the next state of the board.

16. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below. Now after pouring some non-negative integer cups of champagne, return how full the jth glass in the ith row is (both i and j are 0-indexed.) Example 1: Input: poured = 1, query_row = 1, query_glass = 1 Output: 0.00000 Explanation: We poured 1 cup of

champange to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty. Example 2: Input: poured = 2, query_row = 1, query_glass = 1 Output: 0.50000 Explanation: We poured 2 cups of champange to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champange.

Aim

To find how full a glass is after pouring champagne into the tower.

Algorithm

1. Start
2. Create DP table
3. Pour champagne into top glass
4. Distribute excess equally
5. Continue for all rows
6. Print queried glass value
7. Stop **Code** poured = 2 query_row = 1 query_glass = 1 rows = [[0] * (i + 1) for i

in range(102)] rows[0][0] = poured for i in range(100):

for j in range(i + 1):

excess = max(0, rows[i][j] - 1) rows[i +

1][j] += excess / 2 rows[i + 1][j + 1] +=

excess / 2 print(min(1,

rows[query_row][query_glass]))

Input poured =

2 query_row =

1 query_glass =

1

Output

Result

The program successfully calculates the amount of champagne in the queried glass.